1. Introduction

This report presents a Document Question Answering System built using Retrieval-Augmented Generation (RAG). The goal of the system is to allow users to upload PDF documents and ask questions based on their content. The system processes uploaded documents, creates text embeddings, stores them in a vector database, retrieves relevant document chunks, and then uses a Large Language Model (LLM) to generate accurate responses. LangChain, FAISS, HuggingFace embeddings, and Streamlit serve as the core technologies used in this project.

SOURCE CODE:

```python
import streamlit as st
from langchain_community.document_loaders import PyPDFLoader
from langchain.text_splitter import RecursiveCharacterTextSplitter
from langchain_community.embeddings import HuggingFaceEmbeddings
from langchain_community.vectorstores import FAISS
from langchain.chains import RetrievalQA
from langchain_community.llms import HuggingFacePipeline
from transformers import pipeline
import tempfile
import os

# ---------------------------------------------------
# Page Configuration
# ---------------------------------------------------
st.set_page_config(page_title="Document QA using RAG", layout="wide")

st.title("📄 Document Question Answering System using RAG")

st.write("Upload one or more PDF files and ask questions based on the document content.")

# ---------------------------------------------------
# Load Embeddings (Cached)
# ---------------------------------------------------
@st.cache_resource
def load_embeddings():
    return HuggingFaceEmbeddings(
        model_name="all-MiniLM-L6-v2"
    )

# ---------------------------------------------------
# Load LLM (Cached)
# ---------------------------------------------------
@st.cache_resource
def load_llm():
    pipe = pipeline(
        "text2text-generation",
        model="google/flan-t5-base",
        max_length=512,
        temperature=0
    )
    return HuggingFacePipeline(pipeline=pipe)

# ---------------------------------------------------
# File Upload
# ---------------------------------------------------
uploaded_files = st.file_uploader(
    "Upload PDF files",
    type="pdf",
    accept_multiple_files=True
)
```

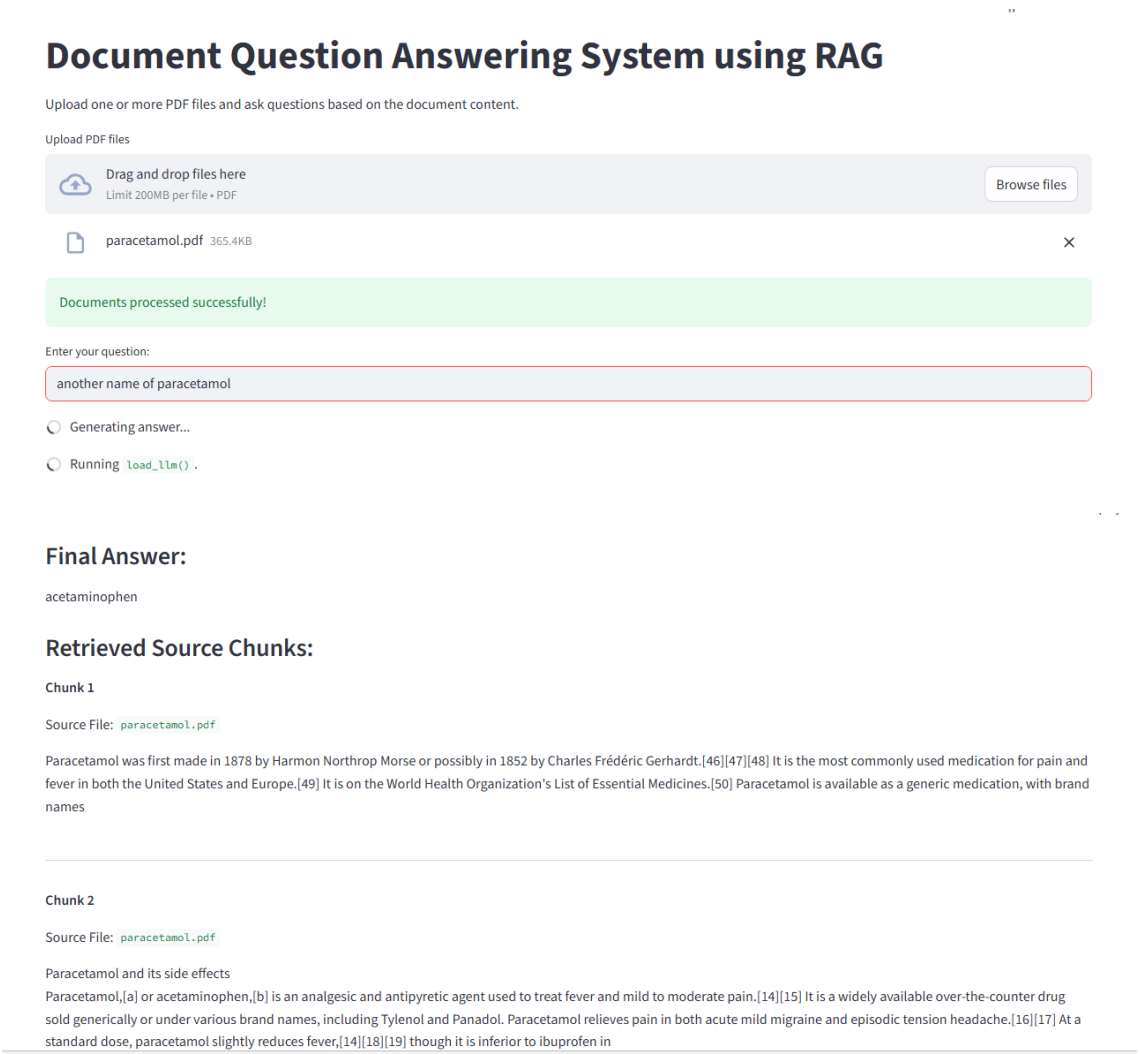
```python
if uploaded_files:

    try:
        all_documents = []

        with st.spinner("Processing documents..."):

            for uploaded_file in uploaded_files:

                # Save temporarily
                with tempfile.NamedTemporaryFile(delete=False) as tmp_file:
                    tmp_file.write(uploaded_file.read())
                    tmp_path = tmp_file.name

                loader = PyPDFLoader(tmp_path)
                documents = loader.load()

                # Add source metadata
                for doc in documents:
                    doc.metadata["source"] = uploaded_file.name

                all_documents.extend(documents)

                os.remove(tmp_path)

```

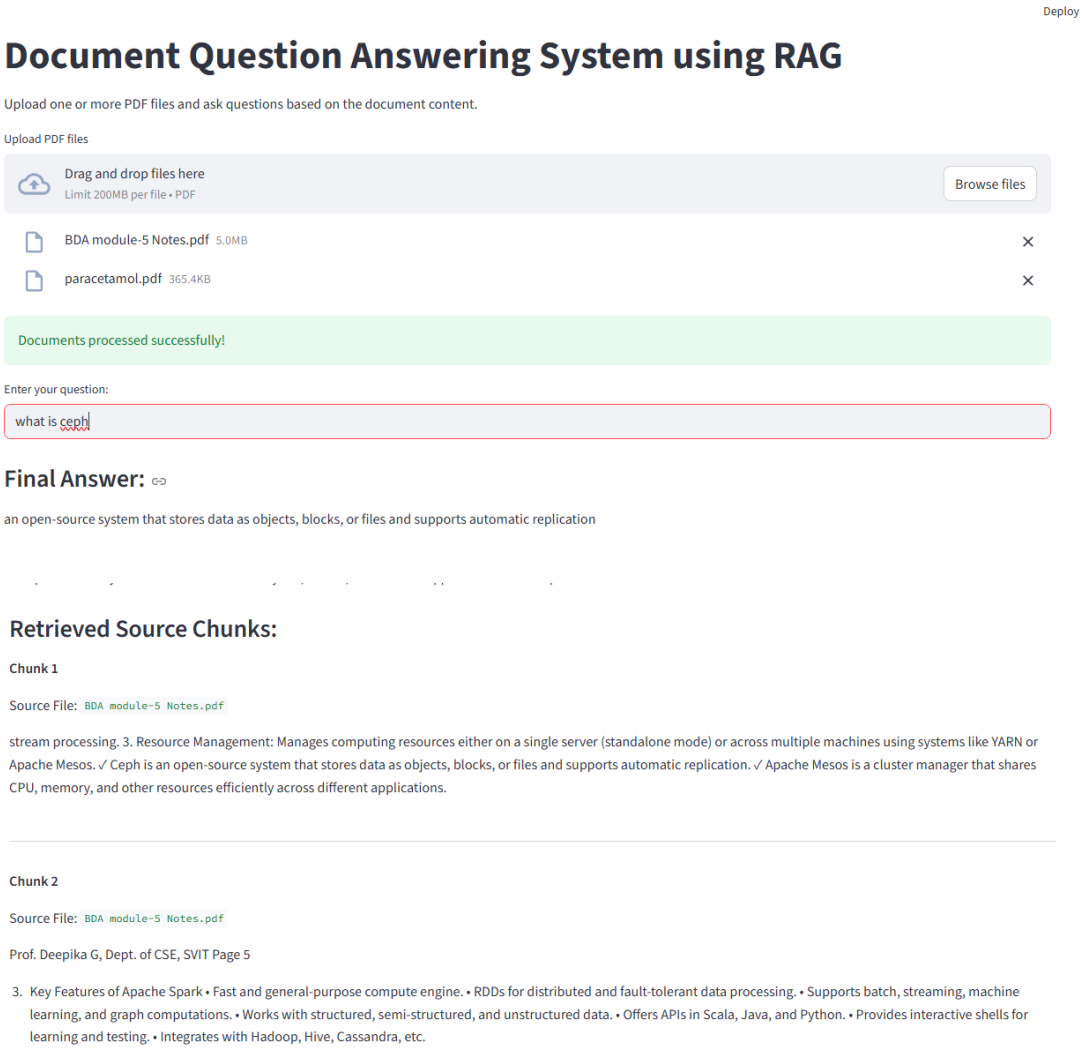
```python
            # ----------------------------------------------
            # Text Splitting
            # ----------------------------------------------
            text_splitter = RecursiveCharacterTextSplitter(
                chunk_size=500,       # As per project explanation
                chunk_overlap=100     # To maintain context
            )

            docs = text_splitter.split_documents(all_documents)

            # ----------------------------------------------
            # Embeddings + FAISS
            # ----------------------------------------------
            embeddings = load_embeddings()

            vectorstore = FAISS.from_documents(
                docs,
                embeddings
            )

        st.success("Documents processed successfully!")

        # ----------------------------------------------
        # Question Input
        # ----------------------------------------------
```

```python
        query = st.text_input("Enter your question:")

        if query:

            with st.spinner("Generating answer..."):

                llm = load_llm()

                retriever = vectorstore.as_retriever(
                    search_kwargs={"k": 3}  # Top-3 retrieval (Required)
                )

                qa_chain = RetrievalQA.from_chain_type(
                    llm=llm,
                    retriever=retriever,
                    return_source_documents=True
                )

                result = qa_chain.invoke({"query": query})

                # ----------------------------------------------
                # Display Answer
                # ----------------------------------------------
                st.subheader(" Final Answer:")
                st.write(result["result"])
```

```python
            # ------------------------------------------------
            # Display Answer
            # ------------------------------------------------
            st.subheader(" Final Answer:")
            st.write(result["result"])

            # ------------------------------------------------
            # Display Source Chunks
            # ------------------------------------------------
            st.subheader("📚 Retrieved Source Chunks:")

            for i, doc in enumerate(result["source_documents"]):
                st.markdown(f"**Chunk {i+1}**")
                st.markdown(f"Source File: `{doc.metadata['source']}`")
                st.write(doc.page_content)
                st.write("---")

    except Exception as e:
        st.error("Something went wrong.")
        st.error(str(e))

else:
    st.info("Please upload at least one PDF file to begin.")
```

OUTPUT:





2. System Architecture

2.1 Components

- Document Loader – Loads and extracts text from PDFs using PyPDFLoader.
- Text Splitter – Splits long documents into manageable chunks using RecursiveCharacterTextSplitter.
- Embedding Generator – Converts text chunks into dense vector representations using all-MiniLM-L6-v2.
- Vector Store (FAISS) – Stores all embeddings for efficient similarity search.
- Retriever – Finds and returns the Top-3 most relevant chunks for a user query.
- LLM (FLAN-T5) – Generates the final coherent answer using retrieved context.
- Streamlit UI – Allows users to upload documents, ask questions, and view results.

2.2 Architecture Flow

1. User uploads PDF files through Streamlit UI.
2. PDFs are read, text extracted, and labeled with metadata.
3. Recursive text chunking prepares the text for embedding.
4. Each chunk is converted into an embedding vector using HuggingFace MiniLM.
5. All embedding vectors are stored in a FAISS index.
6. User enters a question; query is embedded and matched with similar document vectors.
7. Top-3 relevant document chunks are retrieved.
8. FLAN-T5 LLM generates a final answer using retrieved text.
9. System displays the answer and source chunks.

3. Chunking Strategy

Large documents cannot be processed entirely due to LLM context limits. Therefore, RecursiveCharacterTextSplitter is used.

Parameters:

- chunk_size = 500
- chunk_overlap = 100

Justification:

- 500 characters preserve meaningful context.
- 100-character overlap prevents context loss.
- Overlap maintains semantic continuity.
- Recursive splitting ensures clean paragraph-level breaks.

This strategy prepares the text efficiently for embedding and retrieval.

4. Embedding Concept

Embeddings convert text into dense numerical vectors that capture semantic meaning.

Model Used: all-MiniLM-L6-v2 (HuggingFace)

Why Embeddings?

- Enable mathematical comparison of text.
- Capture semantic relationships.
- Support similarity search in FAISS.
- Allow efficient indexing and retrieval.

Role in RAG:

1. Document chunks are embedded.
2. User query is embedded.
3. FAISS computes similarity.
4. Top relevant chunks are retrieved.

Embeddings connect user queries to relevant document content.

5. Retrieval Process

Retrieval provides relevant context to the LLM using **FAISS**.

Steps:

1. User enters a question.
2. Query is converted to embedding.
3. FAISS finds top-3 similar chunks.
4. Retrieved chunks are passed to FLAN-T5.
5. LLM generates final answer.

6. Conclusion

The RAG-based Document QA system successfully integrates document processing, embeddings, FAISS retrieval, and LLM-based answer generation. The Streamlit interface enables easy interaction, while the retrieval pipeline ensures answers are grounded in the uploaded PDFs. This project demonstrates practical implementation of modern NLP and scalable document search systems.